# Cal.com Integration

## 1. Overview

### 1.1 Description

**Cal.com Integration** connects **Cal.com** (open-source scheduling platform) with **Newo Platform**.

It enables:

- Checking real-time availability slots with configurable booking window
- Booking appointments with dynamic custom field support
- Cancelling existing appointments
- Rescheduling appointments using AI-powered extraction from conversation
- Two operation modes: strict availability ("general") and overbooking-friendly ("guest")

This integration is designed for **businesses and service providers** who need **automated appointment scheduling through a conversational AI agent** (voice or chat), including self-hosted Cal.com instances.

### 1.2 Use Cases

- When a client asks about available slots → Check Cal.com availability and return open time slots for the configured booking window
- When a client wants to book an appointment → Collect customer info and custom fields, create booking in Cal.com
- When a client wants to cancel an appointment → Cancel the booking in Cal.com with reason
- When a client wants to reschedule → AI extracts booking details from conversation and reschedules in Cal.com
- When a conversation starts → Silently preload nearest available slots into agent's prompt context

### 1.3 Supported Features

| Feature | Supported | Notes |
|---|---|---|
| Check Availability | ✅ | Configurable booking window (default: 5 days) |
| Book Appointment | ✅ | With dynamic custom booking fields from event type |
| Cancel Appointment | ✅ | With cancellation reason |
| Reschedule Appointment | ✅ | AI-powered payload extraction from conversation |
| Silent Availability Preload | ✅ | Injects slots into prompt on session start |
| General Mode | ✅ | Strict availability — prevents overbooking |
| Guest Mode | ✅ | Allows overbooking, adds manager as guest |
| Personal Event Types | ✅ | Individual calendar events |
| Team Event Types | ✅ | Team-level calendar events |
| Dynamic Custom Fields | ✅ | Auto-discovered from Cal.com event type configuration |
| Self-Hosted Cal.com | ✅ | Configurable base URL |

| | | |
|---|---|---|
| Multi-Location Support | ❌ | One event type per instance; use multiple instances |
| Existing Client Lookup | ❌ | ExistingClientFlow is a placeholder |

## 2. Requirements

Before installation:

- Active **Cal.com** account (hosted or self-hosted)
- **Cal.com API Token** (Bearer token for v2 API)
- Cal.com API base URL (default: `https://api.cal.com/v2` )

## 3. How to Setup

### 3.1 Step 1 — Generate API Token in Cal.com

1. Log in to your **Cal.com** account
2. Navigate to **Settings → Developer → API Keys**
3. Click **Create API Key**
4. Copy the generated token and store it securely

### 3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute | Required | Description |
|---|---|---|
| `calcom_api_token` | ✅ | Bearer token for Cal.com API authentication |
| `calcom_base_url` | ❌ | API base URL (default: `https://api.cal.com/v2`) |
| `calcom_mode` | ❌ | Operation mode: `general` or `guest` (default: `general`) |
| `calcom_event_mode` | ❌ | Event source: `Personal` or `Team` (default: `Personal`) |
| `calcom_event_type` | ❌ | Auto-populated during setup from available event types |
| `calcom_booking_window` | ❌ | Days into future to show slots (default: `5`) |
| `calcom_enable_slot_check` | ❌ | Enable availability checking (default: `True`) |
| `calcom_enable_booking` | ❌ | Enable booking capability (default: `True`) |
| `calcom_enable_cancellation` | ❌ | Enable cancellation capability (default: `True`) |
| `calcom_required_field_descriptions` | ❌ | Descriptions for custom fields (one per line: `field: desc`) |

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Validates the API token
    - Fetches available teams from Cal.com
    - Fetches and selects event types (Personal or Team)
    - Extracts custom booking fields from event type configuration

- Injects dynamic booking schemas for the agent

## 4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

### 4.1 How the Integration Works

The integration operates based on **Triggers and Actions** via the event-driven system.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Cal.com

**Available Triggers**

| Trigger | Event | Description |
|---|---|---|
| **Check Availability** | `get_available_slots` | When the agent needs to look up open slots |
| **Book Appointment** | `book_slot` | When the agent confirms a booking |
| **Cancel Appointment** | `convo_agent_cancel_booking` | When the agent processes a cancellation |
| **Reschedule Appointment** | `reschedule_booking_tool` | When the agent reschedules an existing booking |
| **Session Start** | `session_started` | Silently preloads availability into prompt context |
| **Setup** | `project_publish_finish` | Initializes integration on deploy |

**Available Actions**

- **Get Teams** — Fetches available teams (GET `/teams`)
- **Get Event Types** — Fetches available event types (GET `/event-types`)
- **Get Slots** — Queries available time slots (GET `/slots`)
- **Create Booking** — Books an appointment (POST `/bookings`)
- **Cancel Booking** — Cancels an appointment (POST `/bookings/{id}/cancel`)
- **Reschedule Booking** — Reschedules an appointment (POST `/bookings/{id}/reschedule`)

### 4.2 Architecture Diagrams

**Overall Sequence**

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Cal.com Integration
    participant API as Cal.com API

    Note over I: Setup (on publish)
    A->>I: project_publish_finish
    I->>API: GET /teams
    API-->>I: Teams list
```

```
I->>API: GET /event-types
API-->>I: Event types
I->>API: GET /event-types/(id)
API-->>I: Custom booking fields
I->>A: Booking schemas injected

Note over U,API: Session Start (silent)
A->>I: session_started
I->>API: GET /slots (today, booking_window)
API-->>I: Available slots
I->>A: Inject slots into prompt context

Note over U,API: Availability Check
U->>A: "Any slots this week?"
A->>I: get_available_slots (date, time)
I->>API: GET /slots (eventTypeId, start, end)
API-->>I: Available slots
I->>I: Convert UTC to local timezone
I->>A: result_success (availability[])
A->>U: "Here are the available times..."

Note over U,API: Booking
U->>A: "Book 2:30 PM please"
A->>I: book_slot (customerInfo, dateTime, additional_fields)
I->>I: Convert local time to UTC
alt Guest mode
    I->>I: Add manager as guest
end
I->>API: POST /bookings (attendee, eventTypeId, start)
API-->>I: booking uid
I->>A: result_success (booking_id)
A->>U: "Your appointment is confirmed!"

Note over U,API: Reschedule
U->>A: "Move my appointment to Friday 3 PM"
A->>I: reschedule_booking_tool (memory)
I->>I: LLM extract booking_id + new datetime
I->>I: Convert local time to UTC
I->>API: POST /bookings/(id)/reschedule (start)
API-->>I: Confirmation
I->>A: result_success (rescheduled)
A->>U: "Your appointment has been rescheduled"

Note over U,API: Cancellation
U->>A: "Cancel my appointment"
A->>I: convo_agent_cancel_booking (booking_id)
I->>API: POST /bookings/(id)/cancel (reason)
API-->>I: Confirmation
I->>A: result_success (cancelled)
A->>U: "Your appointment has been cancelled"
```

**AvailabilityFlow**

```
flowchart TD
    E["get_available_slots<br>or session_started"] --> GATE{"Slot
check<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| EX["Extract payload:<br>date, time, silent"]
    EX --> VAL{"Event type<br>configured?"}
    VAL -->|"No"| ERR["ResultError"]
    VAL -->|"Yes"| RANGE["Calculate date range:<br>start → start + booking_window"]
    RANGE --> API["GET /slots<br>{eventTypeId, start, end}"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| TZ["Convert UTC slots<br>to local timezone"]
    TZ --> SILENT{"Silent<br>mode?"}
    SILENT -->|"Yes"| INJECT["Inject into<br>prompt context"]
    SILENT -->|"No"| OUT["result_success<br>{availability[]}"]
```

**BookingFlow**

```
flowchart TD
    E["book_slot"] --> GATE{"Booking<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| EX["Extract payload:<br>customerInfo, dateTime,
<br>additional_fields"]
    EX --> VAL{"Required fields<br>present?"}
    VAL -->|"No"| ERR["ResultError"]
    VAL -->|"Yes"| UTC["Convert local time<br>to UTC"]
    UTC --> MODE{"Guest<br>mode?"}
    MODE -->|"Yes"| GUEST["Add manager<br>as guest"]
    MODE -->|"No"| BUILD["Build booking<br>payload"]
    GUEST --> BUILD
    BUILD --> API["POST /bookings<br>{eventTypeId, start,<br>attendee, guests?,
<br>bookingFieldsResponses}"]
    API --> OK{"HTTP 201?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| OUT["result_success<br>{booking_id: uid}"]
```

**CancellationFlow**

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{"Cancellation<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| VAL{"booking_id<br>present?"}
    VAL -->|"No"| ERR["ResultError"]
    VAL -->|"Yes"| API["POST /bookings/{id}/cancel<br>{cancellationReason}"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| OUT["result_success<br>{cancelled}"]
```

**RescheduleFlow**

```
flowchart TD
    E["reschedule_booking_tool"] --> LOAD["Load bookings<br>from persona"]
    LOAD --> LLM["LLM extract from conversation:<br>booking_id, date, time,
duration"]
    LLM --> MATCH{"Booking<br>matched?"}
    MATCH -->|"No"| ERR["ResultError"]
    MATCH -->|"Yes"| UTC["Convert new time<br>to UTC"]
    UTC --> API["POST /bookings/{id}/reschedule<br>{start, reschedulingReason}"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| OUT["result_success<br>{rescheduled}"]
```

### 4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel) by interacting with the agent. API responses are logged via the integration's HTTP connector.

### 4.4 Testing and Verification

**To test the integration:**

1. **Setup**: Publish the project and verify logs show successful event type discovery and schema injection
2. **Availability**: Ask the agent "What slots are available this week?" — verify slots returned from Cal.com
3. **Booking**: Complete a booking conversation with name, email, phone — verify appointment appears in Cal.com dashboard
4. **Cancellation**: Request cancellation of a booked appointment — verify status changed in Cal.com
5. **Reschedule**: Ask to move an existing appointment — verify new time in Cal.com

**If no action occurs:**

- Ensure `calcom_api_token` is set correctly
- Verify `calcom_base_url` points to the correct Cal.com instance
- Confirm `calcom_event_type` was populated during setup (re-publish if empty)
- Check that the relevant feature flags are enabled ( `calcom_enable_booking` , `calcom_enable_slot_check` , `calcom_enable_cancellation` )
- Verify `calcom_mode` is set to the desired operation mode

> *Note: This integration performs real operations in Cal.com. Appointments created, cancelled, or rescheduled through the agent are reflected in the live Cal.com system.*

## 5. FAQ

**Q: Does the integration import historical appointments?** A: No. Only appointments created after activation are processed. Rescheduling references bookings stored in persona attributes from prior agent interactions.

**Q: Can I connect multiple Cal.com accounts?** A: Each integration instance supports one API token and one event type. For multiple accounts or event types, create separate integration instances.

**Q: What is the difference between "general" and "guest" mode?** A: In **general** mode, the agent checks availability before booking — overbooking is prevented. In **guest** mode, availability checks are bypassed and the manager is added as a guest to the meeting, allowing overbooking.

**Q: How are custom booking fields handled?** A: During setup, the integration auto-discovers required custom fields from the Cal.com event type configuration. These fields are injected into the booking schema so the agent knows to collect them from the user. Field descriptions can be customized via the `calcom_required_field_descriptions` attribute.

**Q: Does it work with self-hosted Cal.com?** A: Yes. Set `calcom_base_url` to your self-hosted instance's API endpoint (e.g., `https://cal.example.com/v2`).

**Q: What happens on session start?** A: If `calcom_enable_slot_check` is enabled, the integration silently fetches today's availability and injects it into the agent's prompt context. This allows the agent to proactively mention available slots without the user asking first.

**Q: What slot durations are supported?** A: Slot duration is determined by the selected Cal.com event type (e.g., 15, 30, 60 min). The `calcom_booking_window` attribute controls how many days ahead to show slots (default: 5 days).